

POCO4A

PROGRAMAÇÃO ORIENTADA

A OBJETOS

Aula 08- Padrões de Projeto
(Factory)

Prof. Rafael G. **Mantovani**

Roteiro



- 1 Introdução
- 2 Padrão: *Factory*
- 3 Exemplo
- 4 Exercícios
- 5 Referências

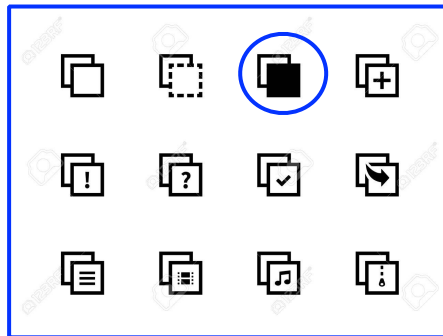
Roteiro

- 1** Introdução
- 2** Padrão: *Factory*
- 3** Exemplo
- 4** Exercícios
- 5** Referências

Introdução

Factory

Padrões de Projeto



"Strategy"



```
34 self.debug = debug
35 self.logger = logging.getLogger(__name__)
36
37 if path:
38     self.file_name = os.path.join(path, "requests.log")
39     self.file_name = self.file_name.replace("\\", "/")
40     self.fingerprints.append(self.file_name)
41
42 @classmethod
43 def from_settings(cls, settings):
44     debug = settings.get("debug", False)
45     return cls(is_debug=debug)
46
47 def request_send(self, request):
48     fp = self.fingerprints.append(request)
49     if fp in self.fingerprints:
50         self.fingerprints.remove(fp)
51     if self.file:
52         self.file.write(fp + os.linesep)
53         self.fingerprints.append(fp)
```

Introdução

No geral, um padrão tem:

- 1 Nome:** descreve o tipo de problema manipulado e sua solução
- 2 Problema:** descreve quando aplicar o padrão
- 3 Solução:** descreve os elementos que corrigem o design, suas relações, responsabilidade e colaborações
- 4 Consequências:** são os resultados de se aplicar o padrão, mostrando os custos (tempo e espaço) e benefícios de se aplicar o padrão

Introdução

Tabela 1.1 O espaço dos padrões de projeto

Escopo	Classe	Propósito		
		De criação	Estrutural	Comportamental
		Factory Method (112)	Adapter (class) (140)	Interpreter (231) Template Method (301)
	Objeto	Abstract Factory (95) Builder (104) Prototype (121) Singleton (130)	Adapter (object) (140) Bridge (151) Composite (160) Decorator (170) Façade (179) Flyweight (187) Proxy (198)	Chain of Responsibility (212) Command (222) Iterator (244) Mediator (257) Memento (266) Observer (274) State (284) Strategy (292) Visitor (305)

Fonte: [Gamma et al, 2000]

Introdução



Objetivo da aula: compreender e aplicar o padrão
Factory

Roteiro



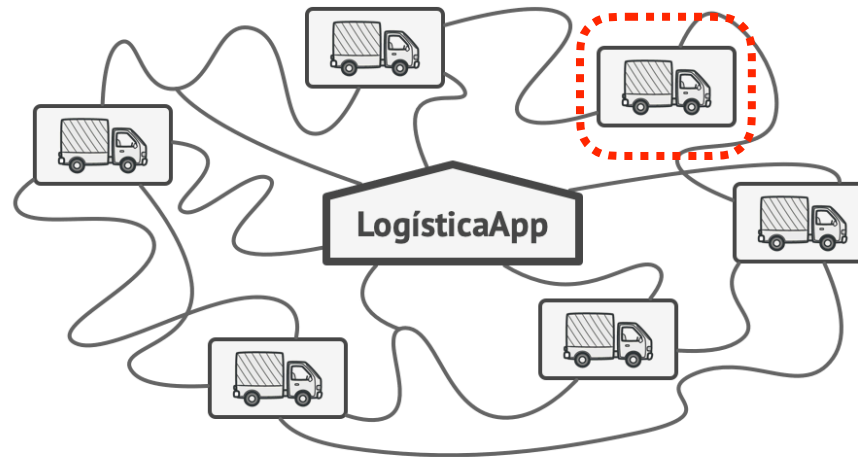
- 1 Introdução
- 2 Padrão: *Factory*
- 3 Exemplo
- 4 Exercícios
- 5 Referências

Padrão: Factory



Empresa de Logística

Padrão: Factory



Empresa de Logística
(Caminhões)

Padrão: Factory



Padrão: Factory

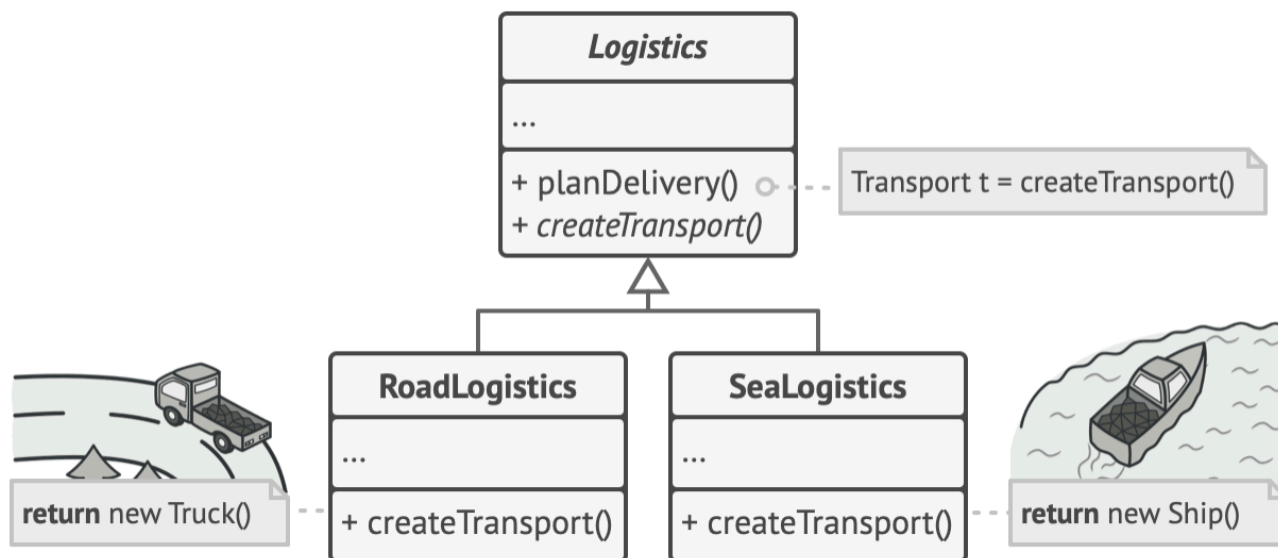
O que fazer?



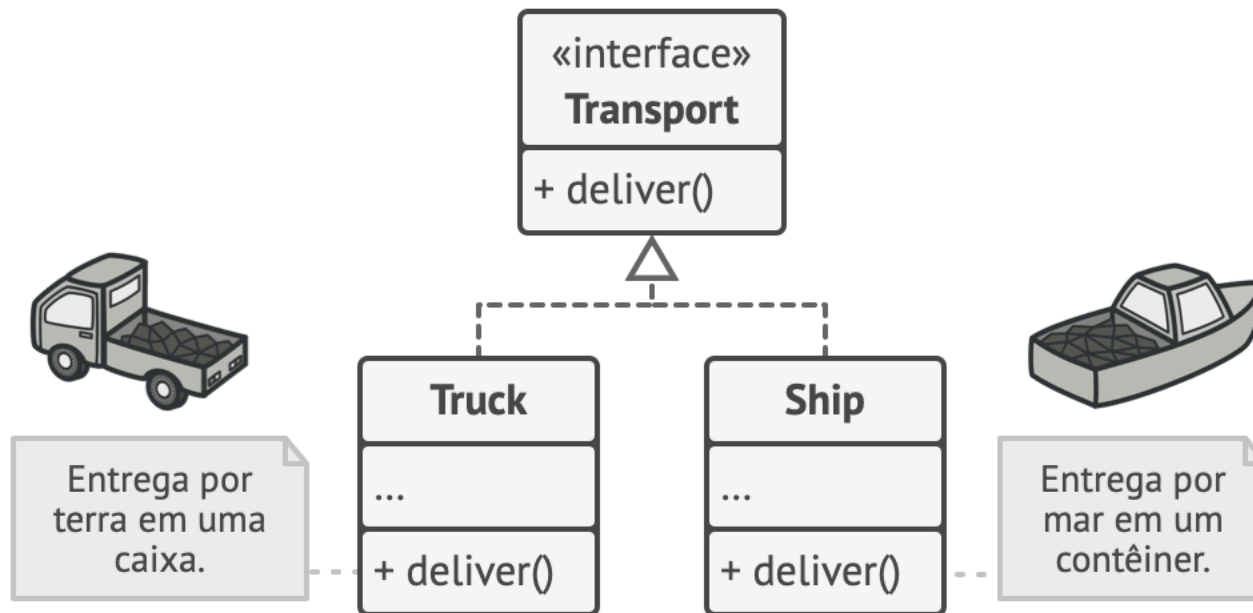
Padrão: Factory

O que fazer?

Factory

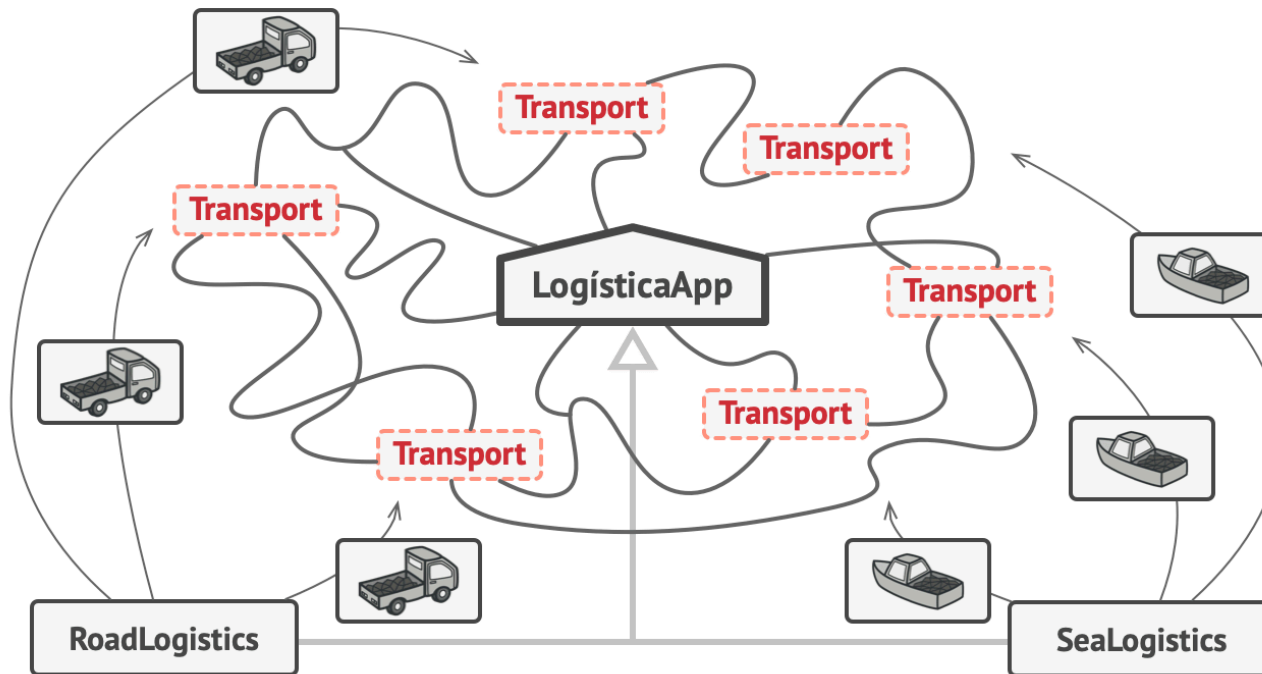


Padrão: Factory



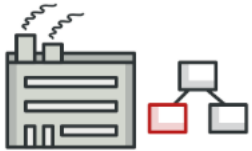
Todos os produtos devem seguir a mesma interface.

Padrão: Factory



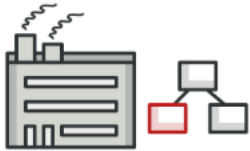
Empresa de Logística v2

Padrão: Factory



Factory

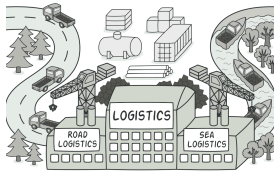
Padrão: Factory



Factory

Define uma **interface** para criar um objeto, mas deixa as **subclasses** **decidirem** qual classe a ser instanciada. O **Factory Method** permite a uma classe postergar a instancição às subclasses.

Padrão: Factory



Factory

Intenção: definir uma interface para criar um objeto, mas deixar as subclasses decidirem que classe instanciar (**Virtual Constructor**)

Padrão: Factory



Aplicabilidade: usamos *Factory Method* quando?

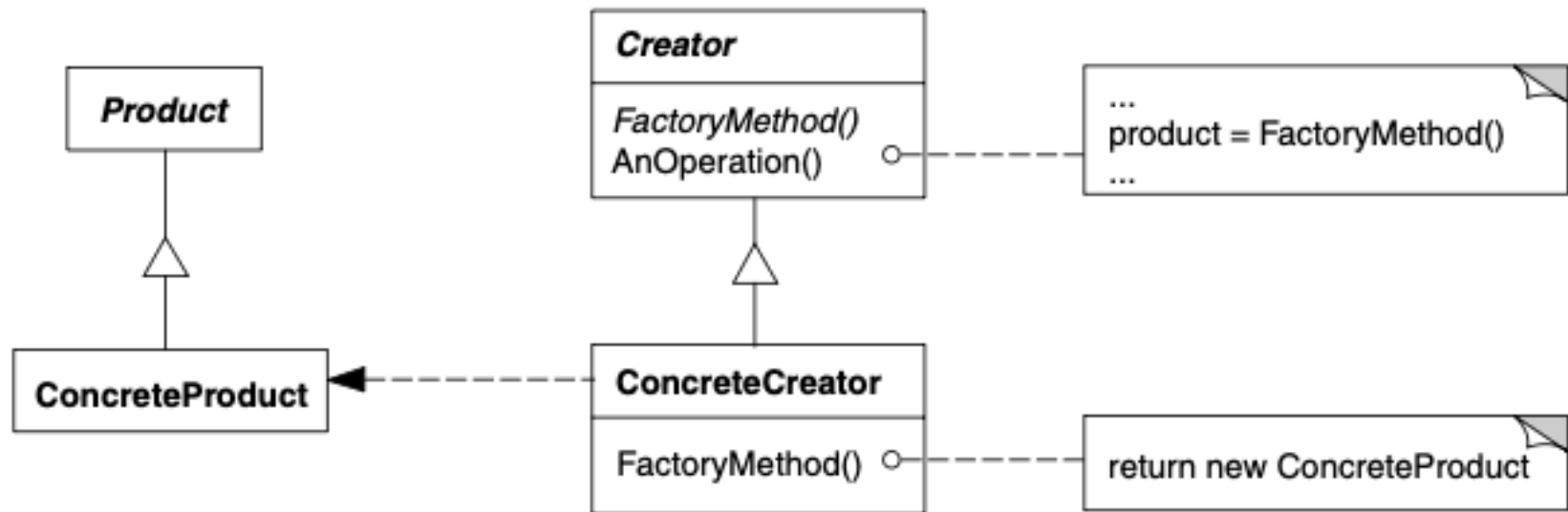
Padrão: Factory

Aplicabilidade: usamos *Factory Method* quando?

- ★ Uma classe não pode antecipar a classe de objetos que deve criar
- ★ Uma classe quer que suas subclasses especifiquem os objetos que criam
- ★ Classes delegam responsabilidade para uma dentre várias subclasses auxiliares, e você quer localizar o conhecimento de qual subclasse auxiliar que é a delegada

Padrão: Factory

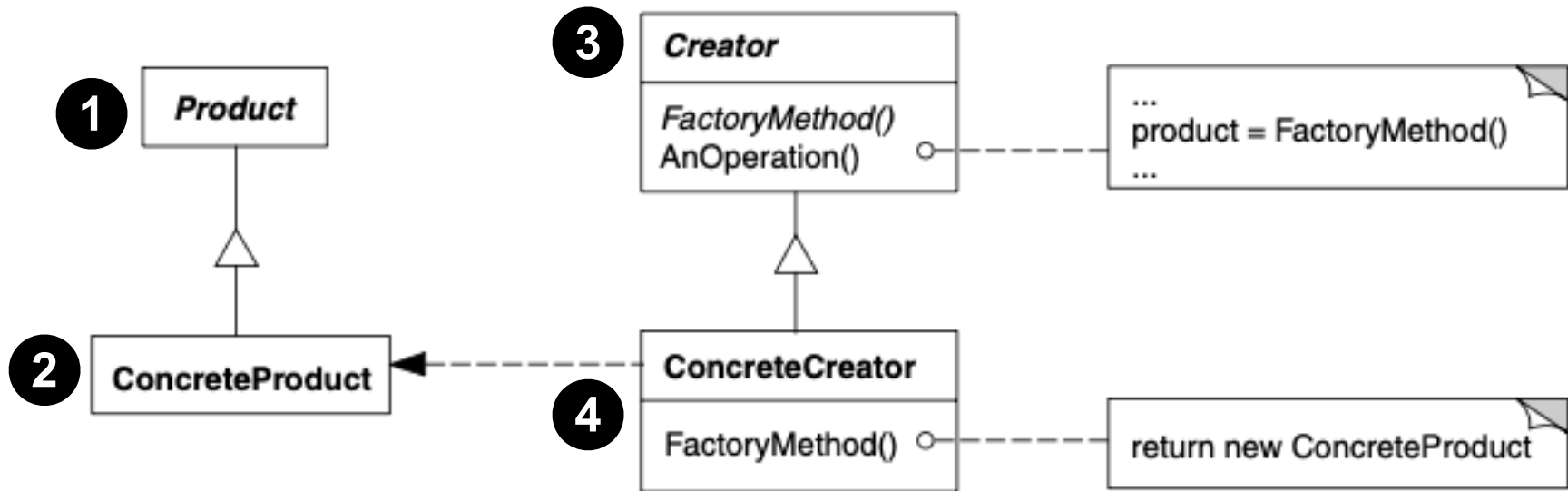
Estrutura:



Fonte: [Gamma et al, 2000]

Padrão: Factory

Estrutura:



Fonte: [Gamma et al, 2000]

Padrão: Factory

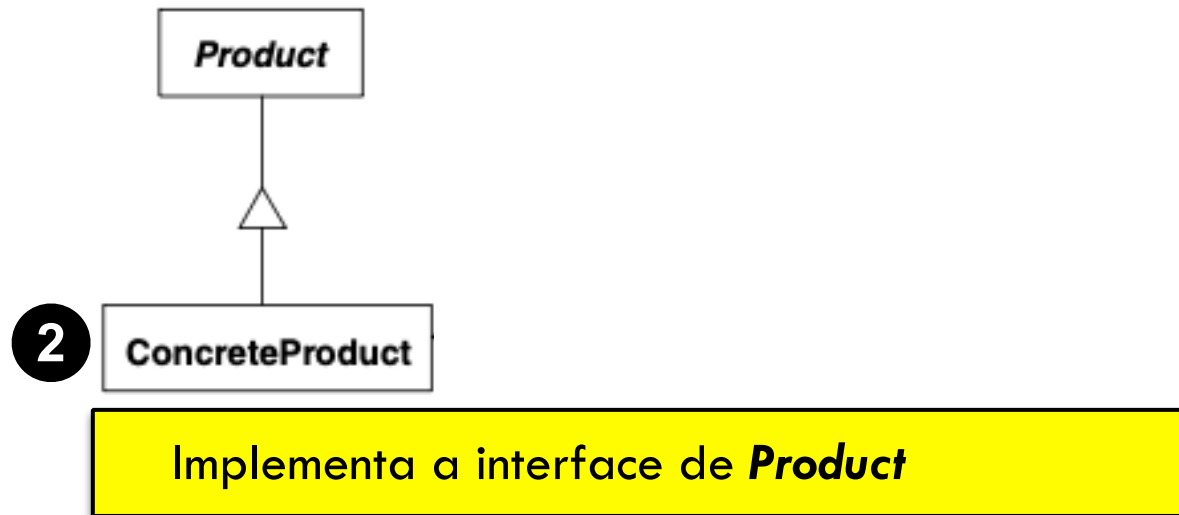
Estrutura:



Define a interface de objetos que o método fábrica cria

Padrão: Factory

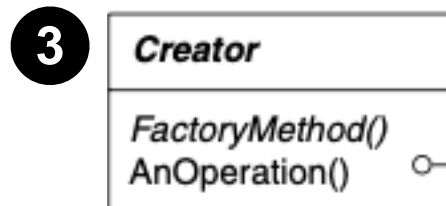
Estrutura:



Fonte: [Gamma et al, 2000]

Padrão: Factory

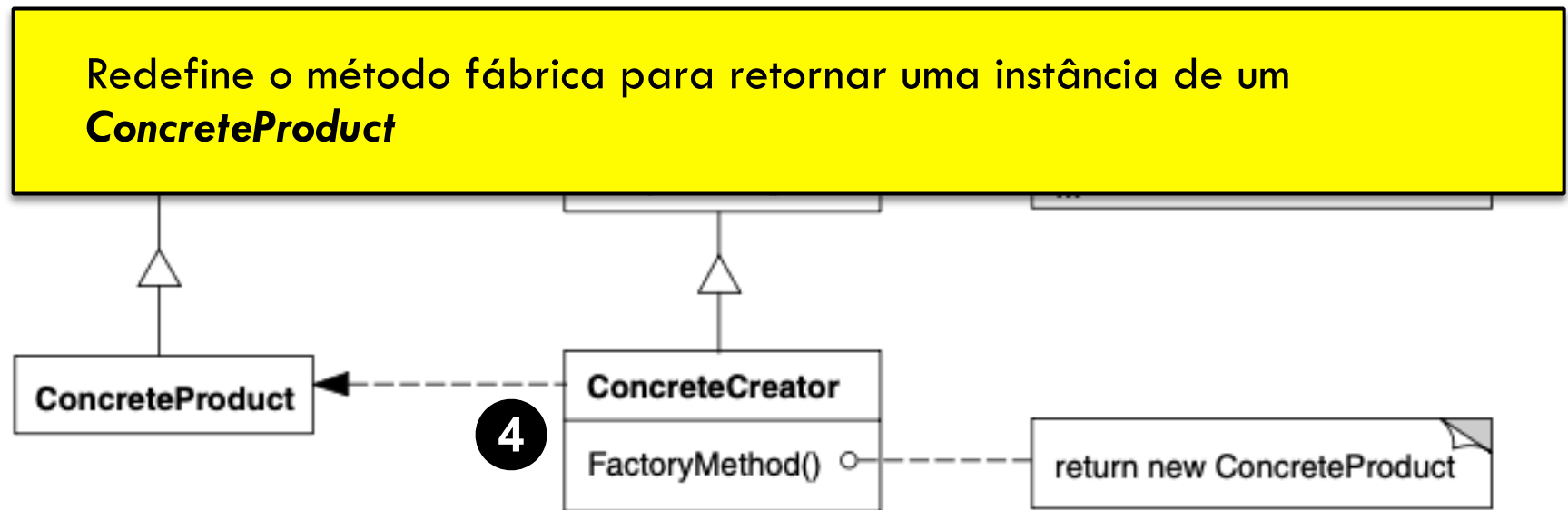
Estrutura:



Declara o método fábrica, o qual retorna um objeto do tipo **Product**. Também define uma implementação por omissão do método factory, que retorna por omissão um objeto **ConcreteProduct**.

Padrão: Factory

Estrutura:



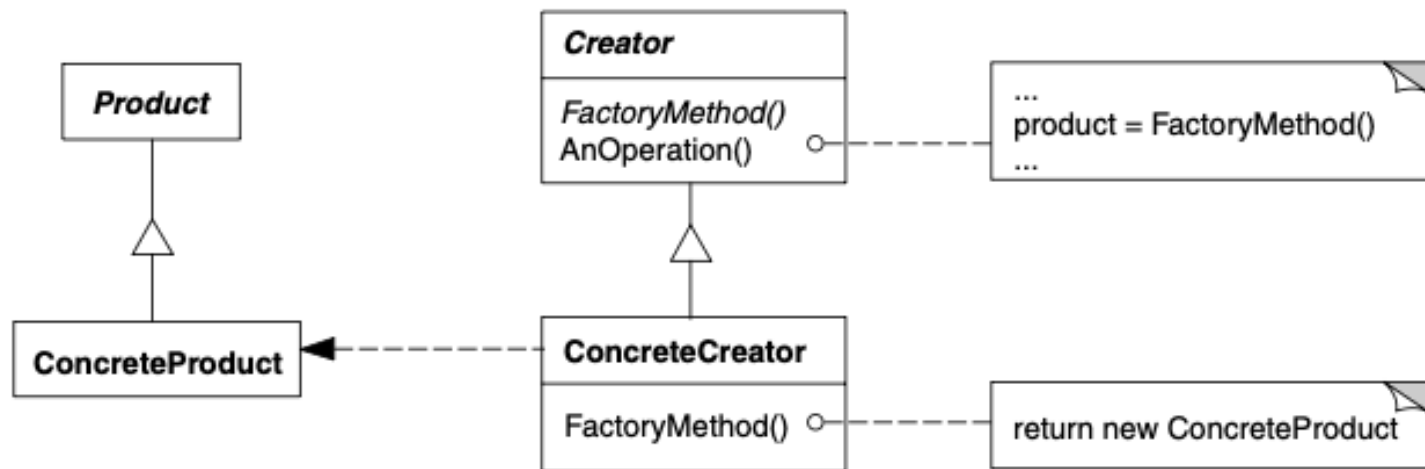
Fonte: [Gamma et al, 2000]

Roteiro



- 1 Introdução
- 2 Padrão: *Factory*
- 3 Exemplo
- 4 Exercícios
- 5 Referências

Exemplo 01



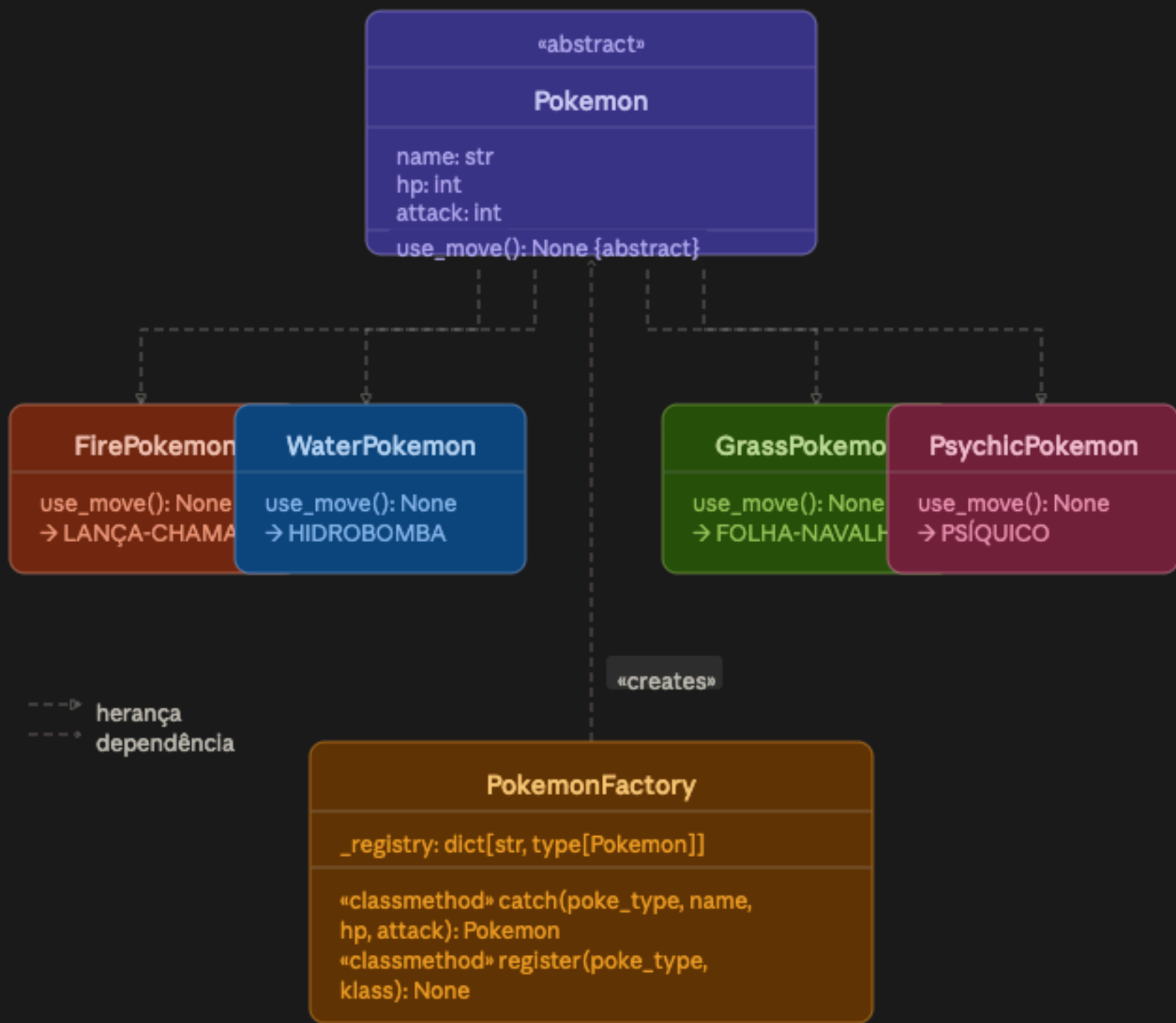
Exemplo 01



Factory.cpp || Factory.py

Exemplo 02: Pokemon





Exemplo 02: Pokemon



Python

```
1. # 1 - Produto Base (abstrato)
2. from abc import ABC, abstractmethod
3.
4. class Pokemon(ABC):
5.     name: str
6.     hp: int
7.     attack: int
8.
9.     @abstractmethod
10.    def use_move(self) -> None: ...
11.
12.    def status(self) -> str:
13.        return (f"{self.name:<14}"
14.                f"HP:{self.hp:>4} "
15.                f"ATK:{self.attack:>4}")
16.
```


Exemplo 02: Pokemon



Python

```
1. # 2 - Produtos Concretos
2. class FirePokemon(Pokemon):
3.     def use_move(self) -> None:
4.         print(f"{self.name} usou LANÇA-CHAMAS! ")
5.         f"dano base: {self.attack}")
6.
7. class WaterPokemon(Pokemon):
8.     def use_move(self) -> None:
9.         print(f"{self.name} usou HIDROBOMBA! ")
10.        f"dano base: {self.attack}")
11.
12. class GrassPokemon(Pokemon):
13.     def use_move(self) -> None:
14.         print(f"{self.name} usou FOLHA-NAVALHA! ")
15.         f"dano base: {self.attack}")
16.
```

Exemplo 02: Pokemon



Python

```
1. # 3 - Factory
2. class PokemonFactory:
3.     _registry: dict[str, type[Pokemon]] = {
4.         "fire": FirePokemon,
5.         "water": WaterPokemon,
6.         "grass": GrassPokemon,
7.     }
8.
9.     @classmethod
10.    def catch(cls, poke_type:str, name:str, hp: int, attack:int)
11.    -> Pokemon:
12.        if poke_type not in cls._registry:
13.            raise ValueError("Esse tipo não existe na Pokedex!")
14.        return cls._registry[poke_type](name=name, hp=hp,
15.            attack=attack)
16.
```

Exemplo 02: Pokemon



Python

```
1. if __name__ == "__main__":
2.     team_data = [
3.         ("fire", "Charizard", 78, 84),
4.         ("water", "Blastoise", 79, 83),
5.         ("grass", "Venusaur", 80, 82),
6.     ]
7.
8.     team = [PokemonFactory.catch(*d) for d in team_data]
9.     for p in team:
10.         print(p.status())
11.
12.     # batalha
13.     for p in team:
14.         p.use_move()
15.
16.
```

Exemplo 02: Pokemon



Python

```
1. if __name__ == "__main__":  
2.     masmorra = Masmorra()  
3.     loja      = Loja()  
4.  
5.     masmorra.derrotar_chefe()  
6.     loja.vender_espolio()    # estado divergente!  
7.
```

Roteiro



- 1 Introdução
- 2 Padrão: *Factory*
- 3 Exemplo
- 4 Exercícios
- 5 Referências

Exercícios



HANDS ON :)))

Exercício 01

- Uma empresa deseja desenvolver um sistema capaz de enviar notificações por diferentes meios de comunicação.
- Inicialmente, o sistema deverá suportar: [E-mail, SMS, Push Notification]
- O sistema deve ser desenvolvido utilizando o padrão de projeto **Factory**, de forma que novas formas de notificação possam ser adicionadas futuramente sem modificar o código principal da aplicação.

Exercício 01

Requisitos

1. Criar uma interface (ou classe abstrata) **Notificacao**.
2. Cada tipo de notificação deve implementar o método:
 - **enviar**(mensagem)
3. Criar uma fábrica responsável por instanciar o tipo correto de notificação.
4. Criar um programa principal que:
 - solicite o tipo de notificação;
 - crie o objeto usando a fábrica;
 - envie uma mensagem.

Exercício 02

- Uma empresa está desenvolvendo um sistema para gerenciamento de meios de transporte urbano. O sistema deve permitir a criação de diferentes veículos:
 - Ônibus
 - Táxi
 - Bicicleta Compartilhada
- Cada tipo de transporte possui um comportamento específico para realizar viagens.
- O sistema deve utilizar o padrão de projeto **Factory Method** para permitir a criação flexível de novos meios de transporte no futuro.

Exercício 02

Requisitos

1. Criar uma interface (ou classe abstrata) **Transporte**.
2. Cada transporte deve implementar o método:
 - **viajar()**
3. Implementar as subclasses: **Onibus**, **Taxi**, **Bicicleta**
4. Criar uma fábrica responsável pela criação dos objetos
5. O programa principal deve:
 - Solicitar o tipo de transporte;
 - Criar o objeto usando a fábrica;
 - Executar a viagem.

Entregável Semanal 08

□ Padrões de Projeto



Entregável Semanal 08

□ Padrões de Projeto

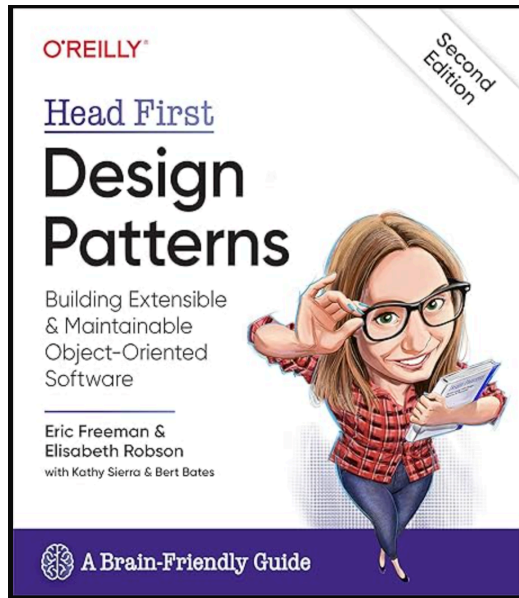
Objetivo: Aplique o padrão Factory para a criação de objetos no jogo. Implemente:

- **ItemFactory** → *criar_arma*(tipo), *criar_armadura*(tipo), *criar_pocao*(tipo)
- **JogadorFactory** → *criar_guerreiro*(nome, raça), *criar_mago*(nome, raça), *criar_ladrao*(nome, raça)
- Testar os padrões criando personagens e itens!

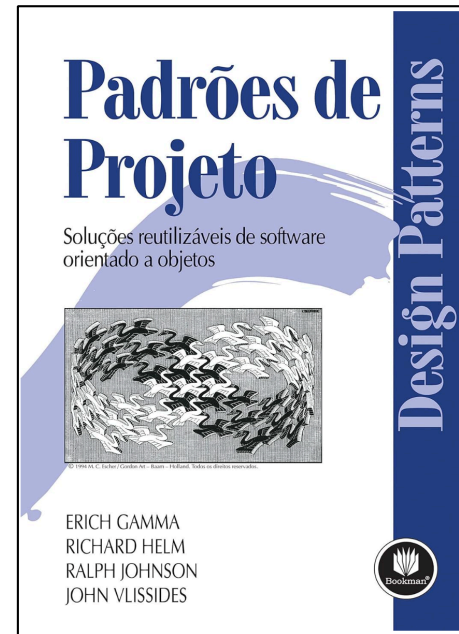
Roteiro

- 1 Introdução
- 2 Padrão: *Factory*
- 3 Exemplo
- 4 Exercícios
- 5 Referências

Referências sugeridas



[Freeman & Robson, 2021]



[Gamma et al, 2000]

Referências online



[Refactoring Guru]



Obrigado :)

Prof. Rafael G. **Mantovani**
rafaelmantovani@utfpr.edu.br